ANSWERS

1. a) $(\overline{A}\,\overline{B}\,C)$. One mark for answer and three for (any) method steps.

   b) See Fig 1.1 for K-map. This gives $Y = \overline{X_3}\,\overline{X_2}\,\overline{X_0} + \overline{X_3}\,X_2\,\overline{X_1} + X_3\,\overline{X_2}\,X_1$

   Must be in SOP form for marks. 1 mark for smallest number of implicants. 3 marks for smallest number of literals in each implicant.

   c) $(A+\overline{D})(\overline{B}+\overline{D})(\overline{C}+\overline{D})$

   Any method is fine (I used a K-map). 2 marks for correct answer. 1 method mark.

   d) $(-3)_{10} = 1111\ 1101 = 1111\ 1111\ 1111\ 1101$

   $(10)_{10} = 0000\ 1010 = 0000\ 0000\ 0000\ 1010$

   1 mark for each correct bit string.

   e) Take state to be (FF2,FF1), then diagram is given in Fig. 1.2. One mark for each state with outedges correctly labelled.

   f) $B\,\overline{C} + \overline{A}\,C + \overline{A}\,B$. Two marks for correct answer. One method mark. Any method OK – taught method is to inspect K-map for non-overlapping adjacent 1s.

   g) Answer:

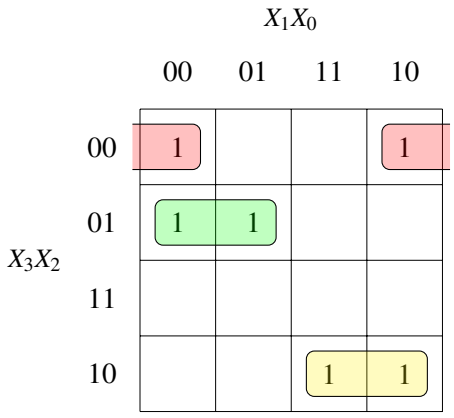


Figure 1.1: K-map

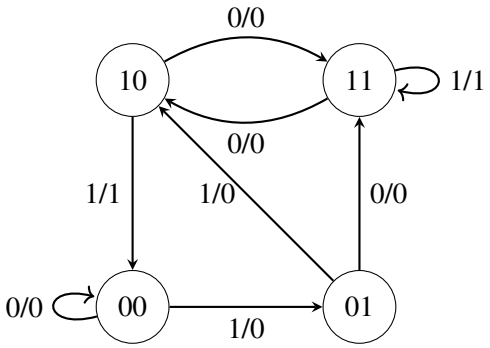


Figure 1.2: Mealy machine diagram.

R0	=	0xFFFF (65535)
R1	=	0x0011 (17)
R2	=	0x0044 (68)
R3	=	0x0032 (50)
R4	=	0x8000 (32768)
R5	=	0xFE00 (65024)
PC	=	0x000B (11)
CODEMEM[0]	=	0x01FF (511)

1 mark for each correct answer.

h)

$$u(X) = 2^{n-1}X(n-1) + \sum_{i=0}^{6} 2^i X(i)$$

$$z(X) = -2^{n-1}X(n-1) + \sum_{i=0}^{6} 2^i X(i)$$

$$\rightarrow u(X) - z(X) = 2^n X(n-1)$$

2 marks for answer, 1 for method.

This means that in n bits, signed and unsigned representations are identical and therefore an n bit binary adder will also implement twos complement signed addition. 1 mark.

i) Answer. NB - C values equal to this answer modulo 2^8 are accepted.

F1	F2	F3	A	B	C
0	0	0	a	b	$a+b$
0	0	1	a	b	$a+b+1$
0	1	0	a	b	$-b+a-1$
0	1	1	a	b	$-b+a$
1	0	0	a	b	$-a+b-1$
1	0	1	a	b	$-a+b$
1	1	0	a	b	$-a-b-2$
1	1	1	a	b	$-a-b-1$

1 mark for each row.

b)

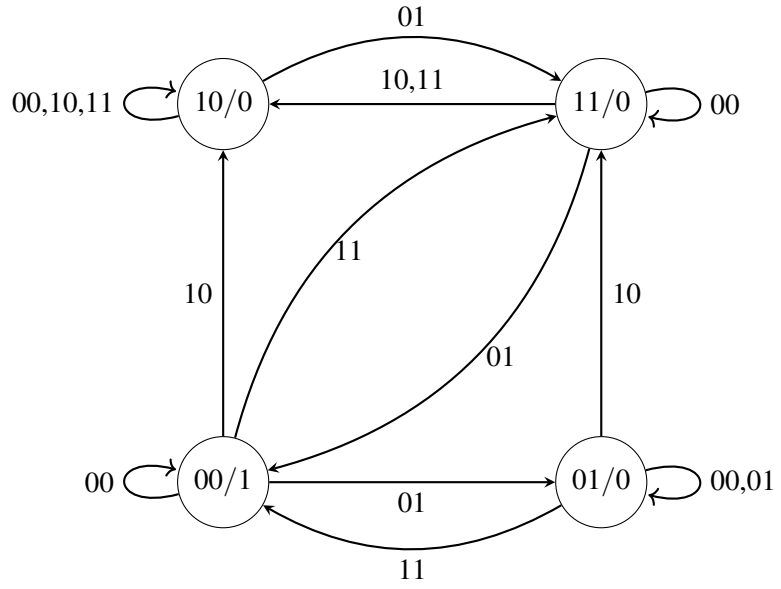


Figure 2.1: Moore machine diagram.

2. a) Writing the states and inputs using their two's complement bit encodings, a suitable diagram is given in Fig. 2.1. Two marks for correct state transitions from each node (8 total). One mark deducted if an error in the output. for any state.

c) K-maps are shown in Fig. 2.2. They may not be unique: in particular, $Y^+(0)$ has a symmetry-breaking implicant. The maps shown give rise to the following expressions:

$Y^+(1) = \overline{X(0)}\,Y(1) + X(1)\,\overline{Y(0)} + X(1)\,\overline{X(0)} + Y(1)\,\overline{Y(0)} + X(1)\,Y(1)$

5 marks. 4 for correct groups, 1 for translation into expression.

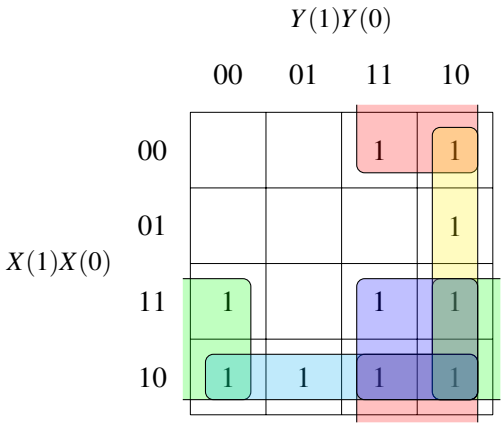
$Y^+(0) = \overline{X(1)}\,\overline{X(0)}\,Y(0) + X(0)\,\overline{Y(1)}\,\overline{Y(0)} + \overline{X(0)}\,\overline{Y(1)}\,Y(0) + \overline{X(1)}\,X(0)\,\overline{Y(0)} + \overline{X(1)}\,X(0)\,\overline{Y(1)}$

5 marks. 4 for correct groups, 1 for translation into expression.

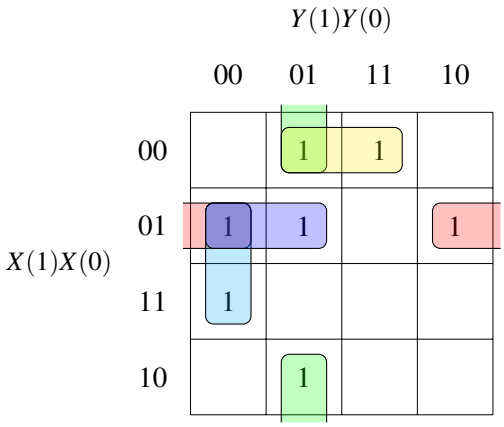
$Z = \overline{Y(1)}\,\overline{Y(0)}$

2 marks. 1 method mark.

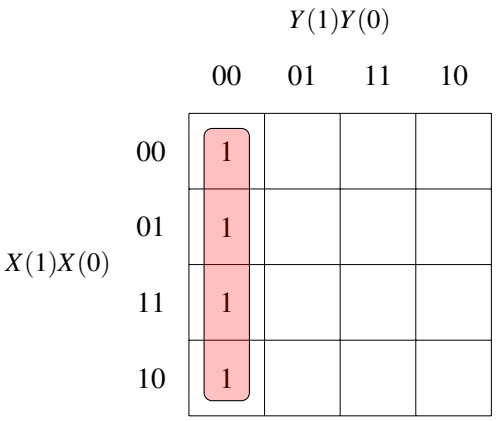
d) Fig. 2.3 illustrates a direct implementation of the SOP expressions, using 2-input NANDs as inverters. 2 marks total for correctly associating D and Q with W^+ and W, respectively for each state variable W. 1 mark each for using NANDs to build inverters and sum terms. 2 marks for correct implementation of the $Y(1)$ expression. 2 marks for the correct implementation of the $Y(0)$ expression. 2 marks for the correct implementation of the Z expression.



(a) $Y^+(1)$



(b) $Y^+(0)$



(c) Z

Figure 2.2: K-maps

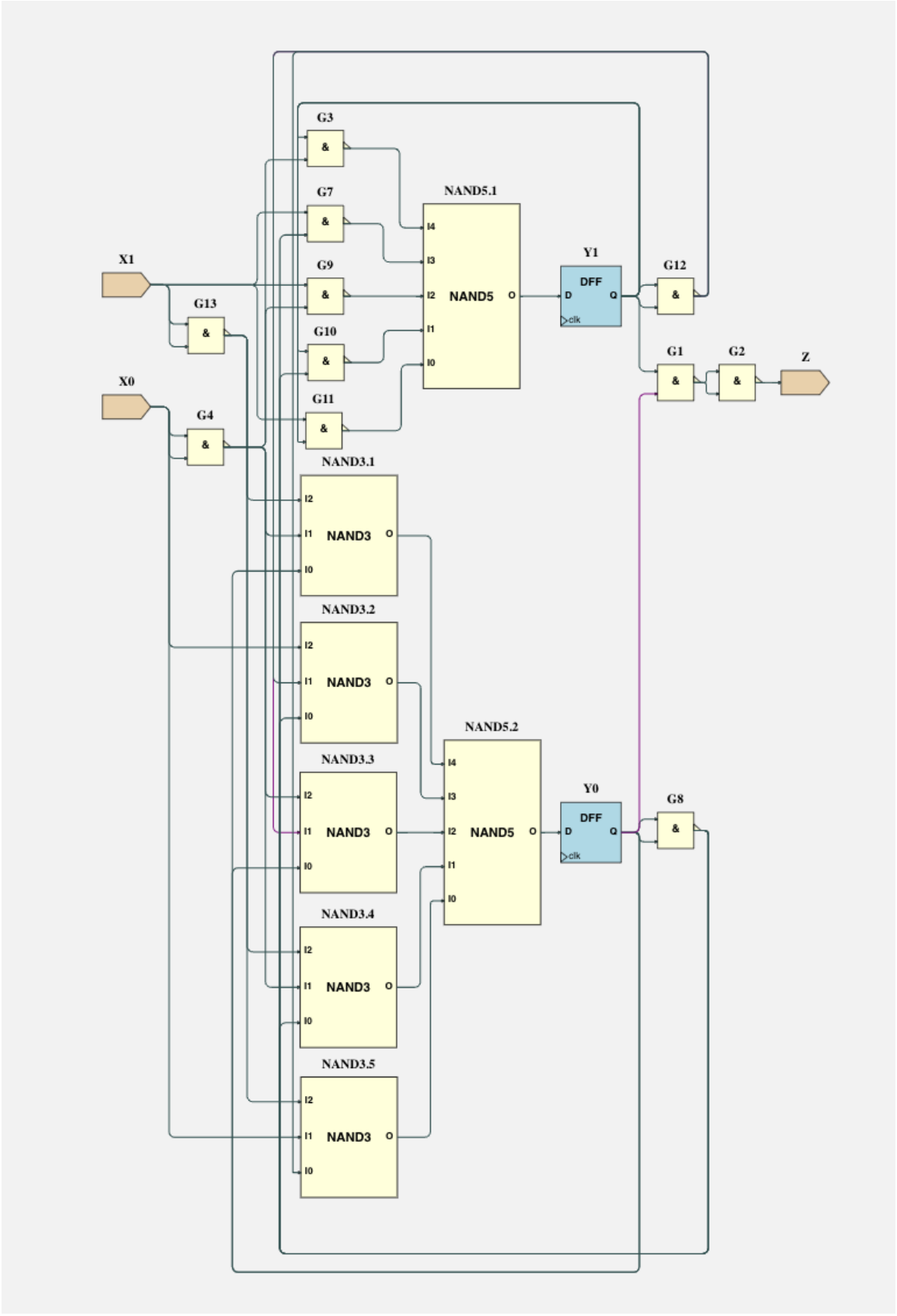


Figure 2.3: FSM implementation

3. a) Answer:

```
LDR R1, [R0]
LDR R2, [R1] // x0
LDR R3, [R1,#1] // x1
LDR R4, [R0,#-1]
LDR R5, [R4] // x3
LDR R4, [R4,#1] // x2
```

2 marks for 6 instructions. 1 mark for 7 instructions. 1 mark each for 4 correct transfers.

b) Set R2 = 0xFFFF, after which a single instruction:

```
SUB Ra, R2, Rb
```

will implement bitwise logical NOT.

Proof this works:
$R2 = \texttt{0xFFFF} \rightarrow R2(i) = 1.$
$1 - R_b(i) = \overline{R_b(i)}$
1 mark for the instruction. 1 mark for reason.

c) $A(i) + B(i) = \overline{\overline{A(i)}\ \overline{B(i)}} \rightarrow$ A OR B = NOT(NOT(A) AND NOT(B))
(DeMorgan's theorem applied to each bit).
1 mark for the correct expression. 1 mark for the reason.

d) Answer:

```
P2  MOV R2, 0xFF // 0xFFFF because of sign extension
    SUB R0, R2, R0 // invert R0
    SUB R1, R2, R1 // invert R1
    AND R0, R0, R1 // OR
    SUB R0, R2, R0 // invert result
    RET // make it a subroutine
```

2 marks for the correct R0 output.
2 marks for RET and no other entry/exit code.

If output is correct:
2 marks for 4 instructions (not counting RET).
1 mark for 5 instructions (not counting RET).

e) Answer:

```
P3  <copy of P1 instructions>
    MOV R0, R2
    MOV R1, R3
    JSR P2
    MOV R1, R4
    JSR P2
    MOV R1, R5
    JSR P2
```

4 marks for correct code.

f) $n = 7\ (P3) + 3 \times 6\ (P2) + 6\ (P1) = 31$
$m = 6$ (6 LDR instructions in P1)
Throughput = fclock * total / (total + stalls) =

$$\frac{200n}{n+m}\text{MHz} = 167.6\text{MHz}$$

2 marks for n (of which 1 for method).
1 mark for m.
2 marks for throughput (of which 1 method).

g) Answer:

```
ENTRY
    SUB R6, #4
    STR R3, [R6, #1]
    STR R4, [R6, #2]
    STR R5, [R6, #3]
    STR R7, [R6, #4]

EXIT
    LDR R3, [R6, #1]
    LDR R4, [R6, #2]
    LDR R5, [R6, #3]
    LDR R7, [R6, #4]
    ADD R6, #4
```

higher	(R7)
	(R5)
	(R4)
lower	(R3)

The EEP1 Subroutine Call Protocol uses a downwards-growing, empty stack pointer, stack, with R6 = SP.
Order of registers on stack does not matter as long as save/restore are symmetric. 2 marks for entry and exit code correct. 2 marks for stack frame diagram consistent with code. 1 mark for entry and exit correctly save and restore a (possibly wrong) set of registers.